

Lab 4

Aron Enström (aroen488), Sergey Volchkov (servo519)

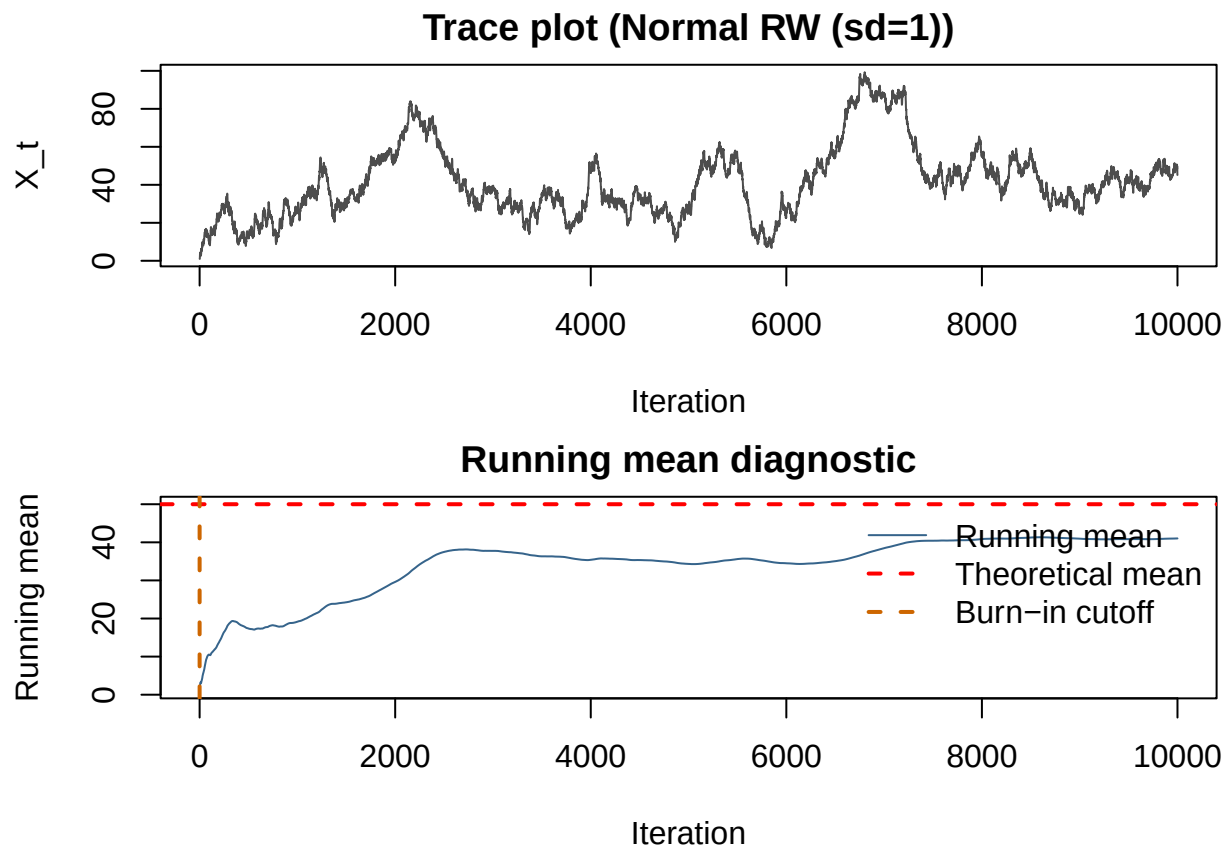
2026-02-11

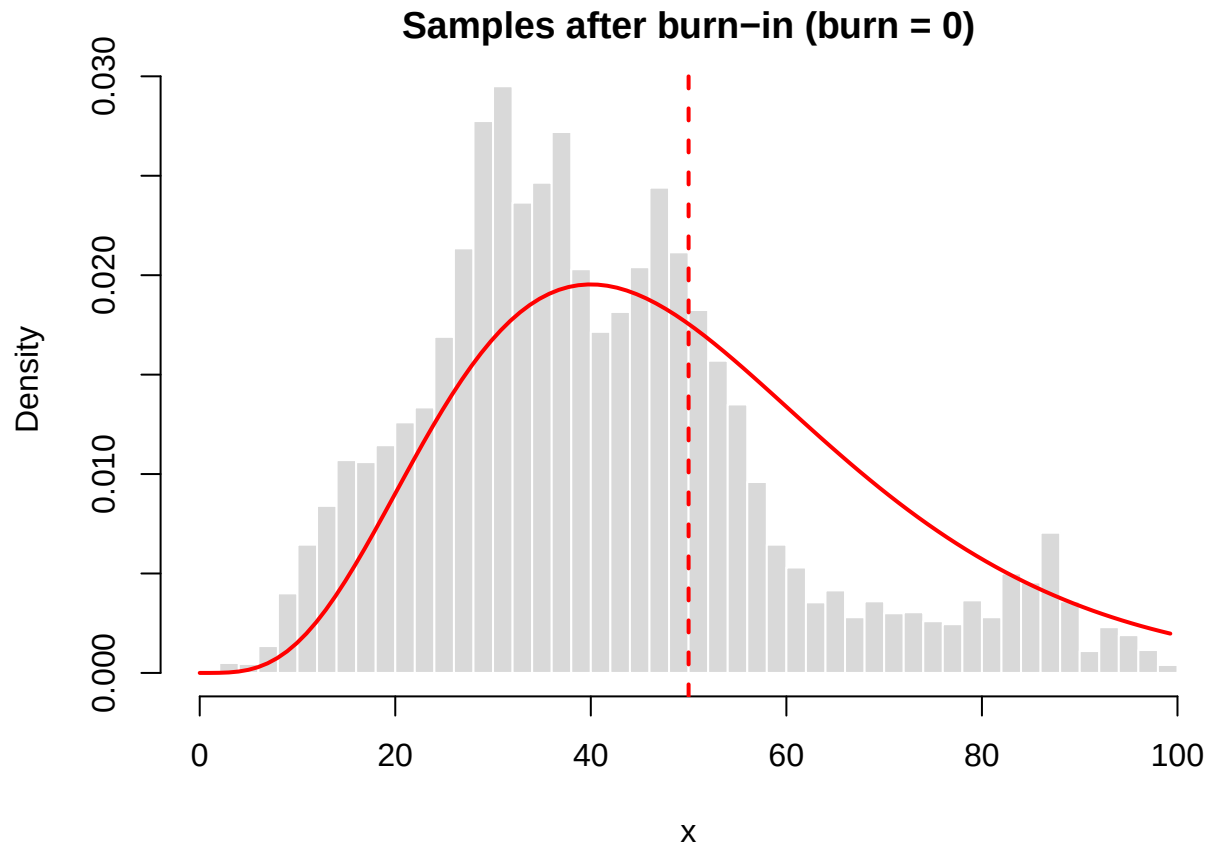
Q1:

a. Metropolis-Hastings with normal distribution as proposal

Proposal: Normal RW (sd=1)

Acceptance rate: 0.9811981

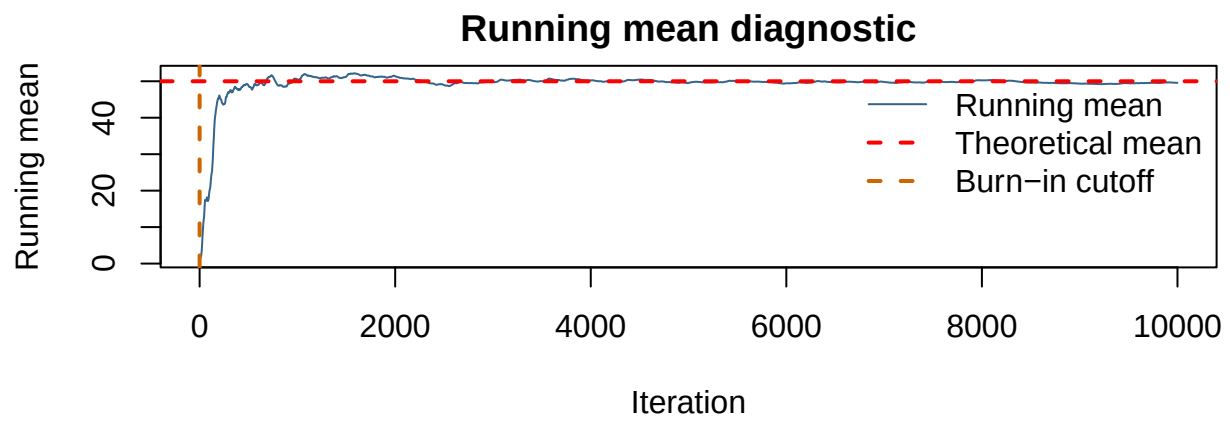
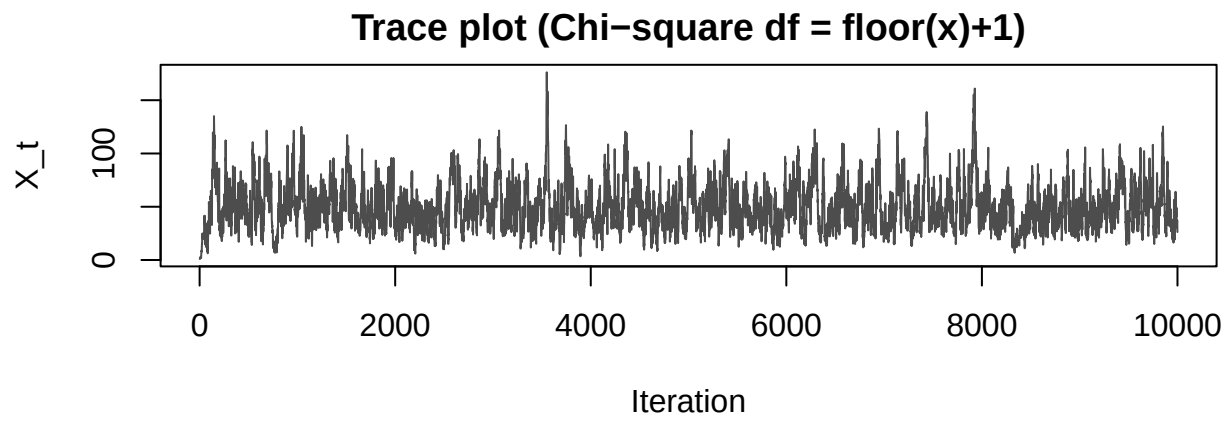


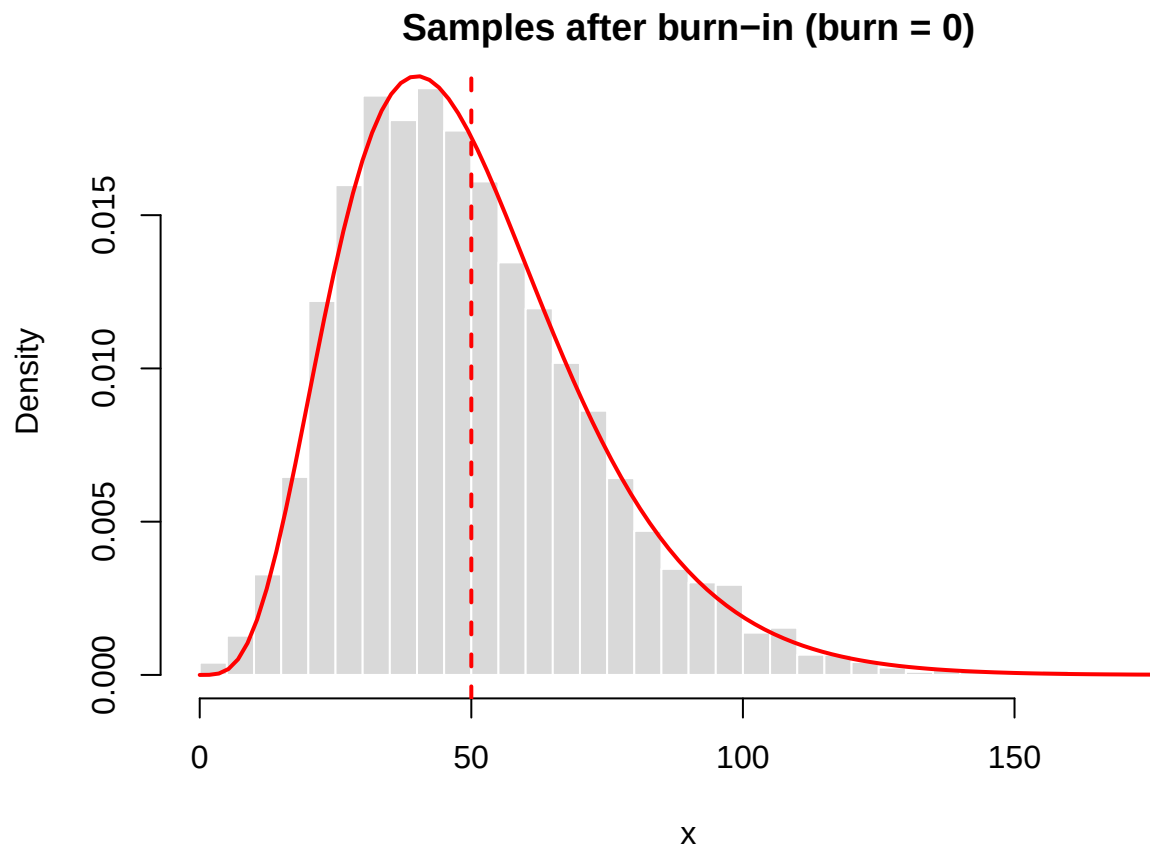


With the standard deviation of 1 and not discarding the samples obtained during the burn-in, we are not seeing convergence to the true mean. The acceptance rate is too high (near 1), which indicates we are moving too slowly.

b. Metropolis-Hastings with chi-square distribution as proposal

```
## Proposal: Chi-square df = floor(x)+1
## Acceptance rate: 0.849985
```

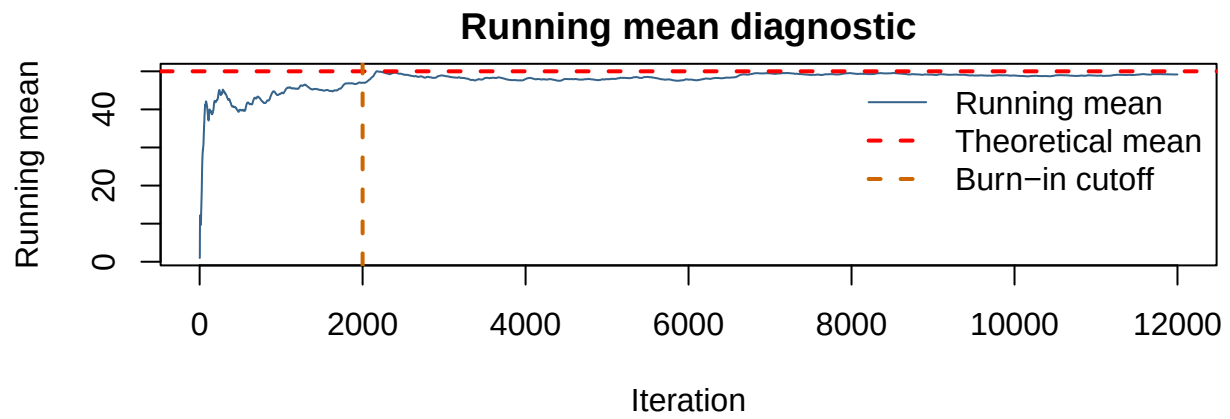
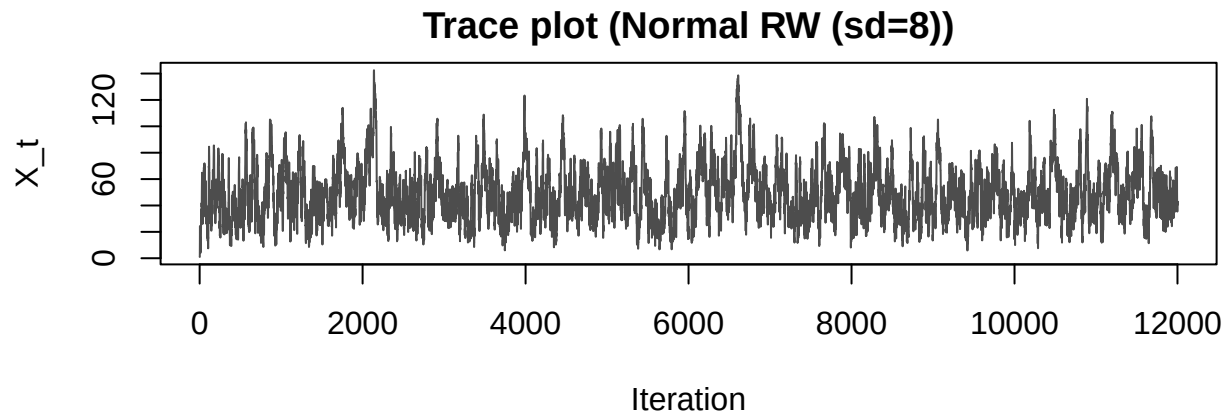


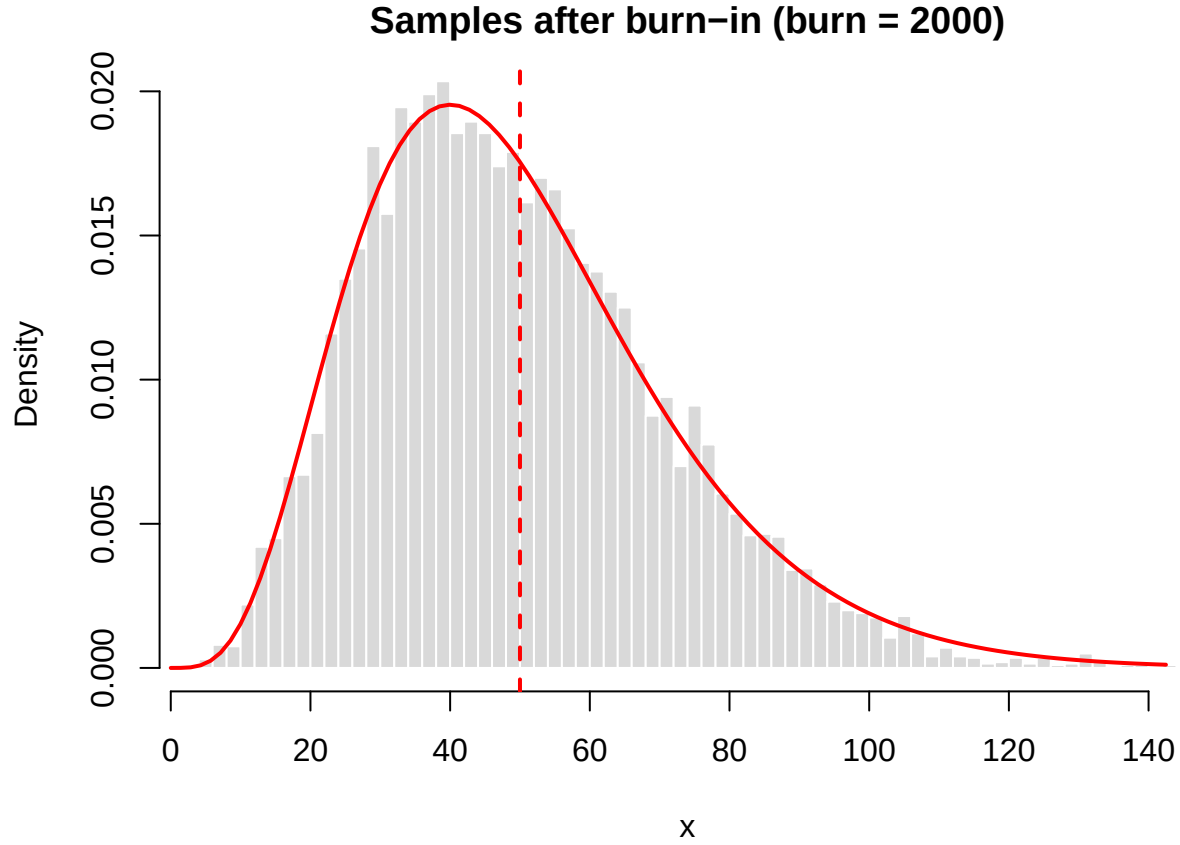


c. Metropolis-Hastings with normal distribution as proposal (sd=8)

Proposal: Normal RW (sd=8)

Acceptance rate: 0.8782399





d. Comparison of the three methods

For $sd = 8$ and the burn-in period of 2000, we are getting much better picture than with the originally used sd of 1. Our resulting histogram matches the gamma distribution much more closely and the acceptance rate is much lower. However, the chi-squared proposal still seems to be the best in terms of the speed of convergence. The chi-square proposal is state-dependent, e.g. $Y \mid X_t = x \sim \chi^2_{\lfloor x \rfloor + 1}$, so its typical proposal size changes with the current location of the chain. This often yields larger, more effective moves when the chain is in higher-density regions away from 0, so the chain can reach and traverse the typical set in fewer iterations than a fixed-scale random-walk. In contrast, a Normal random-walk proposal $Y \mid X_t = x \sim N(x, \sigma^2)$ has a constant step scale σ , which can lead to very slow exploration when σ is too small (even if acceptance is very high).

e. Comparison of $E(X)$ and medians obtained using the three methods

Table 1: Mean and median of MCMC samples

Chain	Mean	Median
Normal RW ($sd=1$)	41.021	37.931
Normal RW ($sd=8$)	49.611	46.872
Chi-square $df=\text{floor}(x)+1$	49.569	46.111

f. Theoretical $E(X)$

For a Gamma distribution parameterized as $X \sim \text{Gamma}(\alpha, \theta)$ (shape α , scale θ), the expectation is $E[X] = \alpha\theta$. Thus for $X \sim \text{Gamma}(5, 10)$, $E[X] = 5 \cdot 10 = 50$ which is the value we used on our plots above.

We can see that the chi-square method gets quite close to it (so does normal with $sd = 8$ but only after discarding the first 2000 samples).

We can also double-check the median in R:

```
## [1] 46.70909
```

Appendix

```
#####
# 1.1
#####

## Target:  $f(x) = (1/(2.4 \cdot 10^6)) \cdot x^4 \cdot \exp(-x/10)$ ,  $x > 0$ 
## This is exactly Gamma(shape=5, scale=10).
shape_true <- 5
scale_true <- 10

## Use log of *unnormalized* density (constant cancels in MH ratio)
log_f_unnorm <- function(x) {
  ifelse(x > 0, 4 * log(x) - x / 10, -Inf)
}

## Generic MH sampler for arbitrary proposal  $q(y/x)$ 
mh_sampler <- function(n = 10000, x0 = 1, rprop, dprop, ...) {
  x <- numeric(n)
  x[1] <- x0
  acc <- 0L

  for (t in 2:n) {
    xt <- x[t - 1]
    y <- rprop(xt, ...)

    log_num <- log_f_unnorm(y) + log(dprop(xt, y, ...)) #  $f(y) q(x/y)$ 
    log_den <- log_f_unnorm(xt) + log(dprop(y, xt, ...)) #  $f(x) q(y/x)$ 
    log_alpha <- log_num - log_den

    if (log(runif(1)) < log_alpha) {
      x[t] <- y
      acc <- acc + 1L
    } else {
      x[t] <- xt
    }
  }

  list(x = x, acc_rate = acc / (n - 1))
}

## Proposal "factories"
make_norm_rw_proposal <- function(sd = 1) {
  list(
    name = sprintf("Normal RW (sd=%g)", sd),
    rprop = function(x) rnorm(1, mean = x, sd = sd),
    dprop = function(y, x) dnorm(y, mean = x, sd = sd)
  )
}
```

```

}

make_chisq_floor_proposal <- function() {
  list(
    name = "Chi-square df = floor(x)+1",
    rprop = function(x) {
      df <- max(1L, floor(x) + 1L)      # floor(x+1) == floor(x)+1
      rchisq(1, df = df)
    },
    dprop = function(y, x) {
      df <- max(1L, floor(x) + 1L)
      dchisq(y, df = df)
    }
  )
}

run_sampler <- function(proposal, burn = 0, n = 10000, x0 = 1) {
  res <- mh_sampler(n = n, x0 = x0, rprop = proposal$rprop, dprop = proposal$dprop)
  x <- res$x

  cat("Proposal:", proposal$name, "\n")
  cat("Acceptance rate:", res$acc_rate, "\n")

  ## Trace + running mean
  par(mfrow = c(2, 1), mar = c(4, 4, 2, 1))

  plot(x, type = "l", col = "gray30", xlab = "Iteration", ylab = "X_t",
       main = paste("Trace plot (", proposal$name, ")", sep = ""))

  run_mean <- cumsum(x) / seq_along(x)
  theo_mean <- shape_true * scale_true
  yl <- range(c(run_mean, theo_mean), finite = TRUE)

  plot(run_mean, type = "l", col = "steelblue4",
       xlab = "Iteration", ylab = "Running mean",
       main = "Running mean diagnostic", ylim = yl)
  abline(h = theo_mean, col = "red", lwd = 2, lty = 2)
  abline(v = burn, col = "darkorange3", lwd = 2, lty = 2)

  legend("topright",
        legend = c("Running mean", "Theoretical mean", "Burn-in cutoff"),
        col = c("steelblue4", "red", "darkorange3"),
        lty = c(1, 2, 2),
        lwd = c(1, 2, 2),
        bty = "n")

  ## Post burn-in sample
  x_post <- if (burn > 0) x[-seq_len(burn)] else x

  ## Histogram + true density overlay
  par(mfrow = c(1, 1), mar = c(4, 4, 2, 1))
  hist(x_post, breaks = 50, freq = FALSE, col = "gray85", border = "white",
       main = sprintf("Samples after burn-in (burn = %d)", burn),

```



```

      xlab = "x")

curve(dgamma(x, shape = shape_true, scale = scale_true),
      from = 0, to = max(x_post),
      add = TRUE, col = "red", lwd = 2)

abline(v = theo_mean, col = "red", lwd = 2, lty = 2)

x_post <- if (burn > 0) x[-seq_len(burn)] else x

invisible(list(
  x = x,
  x_post = x_post,
  acc_rate = res$acc_rate,
  burn = burn,
  proposal = proposal$name
))
}
set.seed(1)
prop_norm <- make_norm_rw_proposal(sd = 1)
out_norm_sd1 <- run_sampler(prop_norm, burn = 0, n = 10000, x0 = 1)
x_norm_sd1 <- out_norm_sd1$x_post
#####
# 1.2
#####
set.seed(1)
prop_chi <- make_chisq_floor_proposal()
out_chi <- run_sampler(prop_chi, burn = 0, n = 10000, x0 = 1)
x_chi <- out_chi$x_post
#####
# 1.3
#####
set.seed(1)
prop_norm <- make_norm_rw_proposal(sd = 8)
out_norm_sd8 <- run_sampler(prop_norm, burn = 2000, n = 12000, x0 = 1)
x_norm_sd8 <- out_norm_sd8$x_post
#####
# 1.5
#####
summ_tab <- data.frame(
  Chain = c("Normal RW (sd=1)", "Normal RW (sd=8)", "Chi-square df=floor(x)+1"),
  Mean = c(mean(x_norm_sd1, na.rm = TRUE),
            mean(x_norm_sd8, na.rm = TRUE),
            mean(x_chi, na.rm = TRUE)),
  Median = c(median(x_norm_sd1, na.rm = TRUE),
              median(x_norm_sd8, na.rm = TRUE),
              median(x_chi, na.rm = TRUE))
)

## Print nicely in R Markdown
knitr::kable(summ_tab, digits = 3, caption = "Mean and median of MCMC samples")
#####
# 1.6

```

```
#####  
qgamma(0.5, shape=5, scale=10)
```